



PlaySBC

Product and Administration Guide

Features, architecture, deployment, operations, regression, and evidence

Version 3.0.0

Public v3.0.0 and private commercial build tracks

Contributor

Sudheer Kumar Vatrapu

Contents

Contents	2
Document Control	2
Common Local Preparation	3
Public v3.0.0 Release: Upgrade And Full Regression	4
Public Source: Build, Package, Upgrade, And Full Regression	5
Private Commercial Build, Upgrade, And Full Regression	6
Focused RFC 5359 Regression	7
Evidence, Prometheus, And Grafana	8
Troubleshooting	9
Detailed Runbooks	10

Document Control

Track	Repository	Artifact identity
Public release	PlaySBC	Published 3.0.0 images and chart
Public source	PlaySBC	Local commit SHA
Private commercial	PlaySBC-Commercial	playsbc-commercial-dev:<commit>

This curated guide does not embed the standalone Markdown runbooks. They remain maintained independently under docs/. The browser edition provides COPY buttons; PDF viewers keep command text selectable but cannot access the system clipboard.

Common Local Preparation

COMMAND

SELECT + COPY

```
set -euo pipefail
export KUBE_CONTEXT=kind-playsbc
export KIND_CLUSTER=playsbc
open -a Docker
until docker info >/dev/null 2>&1; do echo "Waiting for Docker Desktop..."; sleep 5; done
if ! kind get clusters | grep -qx "$KIND_CLUSTER"; then
  kind create cluster --name "$KIND_CLUSTER" --wait 180s
fi
kubectrl --context "$KUBE_CONTEXT" get --raw=/readyz
kubectrl --context "$KUBE_CONTEXT" create namespace playsbc --dry-run=client -o yaml \
| kubectrl --context "$KUBE_CONTEXT" apply -f -
```

Public v3.0.0 Release: Upgrade And Full Regression

Run this entire block after the v3.0.0 chart and four public images are published. It upgrades PlaySBC, RTPengine, Prometheus, and Grafana, verifies every rollout, and launches the 78-profile full-regression catalog.

COMMAND	SELECT + COPY
<pre>cd /Users/sudheerkumar/Documents/Codex/2026-05-18/Mini-Call-Server export PLAYSBC_VERSION=3.0.0 export KUBE_CONTEXT=kind-playsbc helm upgrade --install playsbc \ "https://github.com/sudheerkumarvatrapu/PlaySBC/releases/download/v\${PLAYSBC_VERSION}/playsbc-\${PLAYSBC_VERSION}.tgz" \ --kube-context "\$KUBE_CONTEXT" --namespace playsbc --create-namespace \ --atomic --wait --timeout 10m \ -f configs/kubernetes/active-active-values.yaml \ --set image.repository=ghcr.io/sudheerkumarvatrapu/playsbc \ --set-string image.tag="\$PLAYSBC_VERSION" --set image.pullPolicy=Always \ --set rtpengine.enabled=true \ --set rtpengine.image.repository=ghcr.io/sudheerkumarvatrapu/playsbc-rtpengine \ --set-string rtpengine.image.tag="\$PLAYSBC_VERSION" \ --set rtpengine.image.pullPolicy=Always --set observability.enabled=true kubectl --context "\$KUBE_CONTEXT" -n playsbc get pods -o wide for WORKLOAD in \ statefulset/playsbc-playsbc \ statefulset/playsbc-playsbc-rtpengine \ deployment/playsbc-playsbc-prometheus \ deployment/playsbc-playsbc-grafana; do kubectl --context "\$KUBE_CONTEXT" -n playsbc rollout status "\$WORKLOAD" --timeout=240s done</pre>	
COMMAND (CONTINUED)	SELECT + COPY
<pre>PYTHONPYCACHEPREFIX=/private/tmp/playsbc-public-pycache \ python3 tools/run_k8s_regression_job.py \ --all-profiles \ --runner-image "ghcr.io/sudheerkumarvatrapu/playsbc-k8s-regression:\${PLAYSBC_VERSION}" \ --sipp-image "ghcr.io/sudheerkumarvatrapu/playsbc-sipp:\${PLAYSBC_VERSION}" \ --playsbc-image "ghcr.io/sudheerkumarvatrapu/playsbc:\${PLAYSBC_VERSION}" \ --rtpengine-image "ghcr.io/sudheerkumarvatrapu/playsbc-rtpengine:\${PLAYSBC_VERSION}" \ --set-playsbc-image --set-rtpengine-image \ --no-load-playsbc-image --no-load-rtpengine-image --no-load-sipp-image \ --kind-cluster "\$KIND_CLUSTER"</pre>	

Public Source: Build, Package, Upgrade, And Full Regression

Use this complete workflow before the release artifacts exist:

COMMAND	SELECT + COPY
<pre> set -euo pipefail cd /Users/sudheerkumar/Documents/Codex/2026-05-18/Mini-Call-Server git switch main && git pull --ff-only origin main test -z "\$(git status --porcelain)" export KUBE_CONTEXT=kind-playsbc export KIND_CLUSTER=playsbc export SOURCE_TAG=\$(git rev-parse --short=12 HEAD) export PLAYSBC_REPOSITORY=playsbc-public-dev export RTPENGINE_REPOSITORY=playsbc-rtpengine-public-dev export PLAYSBC_IMAGE="\${PLAYSBC_REPOSITORY}:\${SOURCE_TAG}" export RTPENGINE_IMAGE="\${RTPENGINE_REPOSITORY}:\${SOURCE_TAG}" export PACKAGE_DIR=\$(mktemp -d /private/tmp/playsbc-public-package.XXXXXX) export CHART_VERSION=\$(awk '/^version:/ {print \$2; exit}' charts/playsbc/Chart.yaml) export PLAYSBC_CHART="\${PACKAGE_DIR}/playsbc-\${CHART_VERSION}.tgz" open -a Docker until docker info >/dev/null 2>&1; do echo "Waiting for Docker Desktop..."; sleep 5; done if ! kind get clusters grep -qx "\$KIND_CLUSTER"; then kind create cluster --name "\$KIND_CLUSTER" --wait 180s fi kubectl --context "\$KUBE_CONTEXT" create namespace playsbc --dry-run=client -o yaml \ </pre>	
COMMAND (CONTINUED)	SELECT + COPY
<pre> kubectl --context "\$KUBE_CONTEXT" apply -f - docker build -f docker/playsbc.Dockerfile -t "\$PLAYSBC_IMAGE" . docker build -f docker/rtpengine.Dockerfile -t "\$RTPENGINE_IMAGE" . kind load docker-image "\$PLAYSBC_IMAGE" "\$RTPENGINE_IMAGE" --name "\$KIND_CLUSTER" helm lint charts/playsbc helm package charts/playsbc --destination "\$PACKAGE_DIR" helm upgrade --install playsbc "\$PLAYSBC_CHART" \ --kube-context "\$KUBE_CONTEXT" --namespace playsbc --create-namespace \ --reuse-values --atomic --wait --timeout 10m \ -f configs/kubernetes/active-active-values.yaml \ --set image.repository="\$PLAYSBC_REPOSITORY" \ --set-string image.tag="\$SOURCE_TAG" --set image.pullPolicy=IfNotPresent \ --set rtpengine.enabled=true \ --set rtpengine.image.repository="\$RTPENGINE_REPOSITORY" \ --set-string rtpengine.image.tag="\$SOURCE_TAG" \ --set rtpengine.image.pullPolicy=IfNotPresent --set rtpengine.hostNetwork=false \ --set playsbc.config.media_backend=rtpengine \ --set-string playsbc.config.rtpengine_url=udp://playsbc-playsbc-rtpengine:2223 \ --set observability.enabled=true </pre>	
COMMAND (CONTINUED)	SELECT + COPY
<pre> kubectl --context "\$KUBE_CONTEXT" -n playsbc rollout restart \ statefulset/playsbc statefulset/playsbc-playsbc-rtpengine \ deployment/playsbc-playsbc-prometheus deployment/playsbc-playsbc-grafana for WORKLOAD in \ statefulset/playsbc-playsbc \ statefulset/playsbc-playsbc-rtpengine \ deployment/playsbc-playsbc-prometheus \ deployment/playsbc-playsbc-grafana; do kubectl --context "\$KUBE_CONTEXT" -n playsbc rollout status "\$WORKLOAD" --timeout=240s done kubectl --context "\$KUBE_CONTEXT" -n playsbc get pods -o wide PYTHONPYCACHEPREFIX=/private/tmp/playsbc-public-pycache \ python3 tools/run_k8s_regression_job.py \ --all-profiles \ --playsbc-image "\$PLAYSBC_IMAGE" --rtpengine-image "\$RTPENGINE_IMAGE" \ --set-playsbc-image --set-rtpengine-image \ --no-load-playsbc-image --no-load-rtpengine-image \ --build-runner-image --build-sipp-image --kind-load-images \ --kind-cluster "\$KIND_CLUSTER" </pre>	

Private Commercial Build, Upgrade, And Full Regression

This track creates local development artifacts only; it does not publish a commercial release.

COMMAND	SELECT + COPY
<pre> cd /Users/sudheerkumar/Documents/Codex/2026-05-18/PlaySBC-Commercial git switch main && git pull --ff-only origin main test -z "\$(git status --porcelain)" export SOURCE_TAG=\$(git rev-parse --short=12 HEAD) export PLAYSBC_REPOSITORY=playsbc-commercial-dev export PLAYSBC_IMAGE="\${PLAYSBC_REPOSITORY}:\${SOURCE_TAG}" docker build -f docker/playsbc.Dockerfile -t "\$PLAYSBC_IMAGE" . kind load docker-image "\$PLAYSBC_IMAGE" --name playsbc export PACKAGE_DIR=\$(mktemp -d /private/tmp/playsbc-commercial-package.XXXXXX) export CHART_VERSION=\$(awk '/^version:/ {print \$2; exit}' charts/playsbc/Chart.yaml) export PLAYSBC_CHART="\${PACKAGE_DIR}/playsbc-\${CHART_VERSION}.tgz" helm lint charts/playsbc helm package charts/playsbc --destination "\$PACKAGE_DIR" helm upgrade --install playsbc "\$PLAYSBC_CHART" --kube-context kind-playsbc \ --namespace playsbc --create-namespace --reuse-values --atomic --wait --timeout 10m \ --set image.repository="\$PLAYSBC_REPOSITORY" --set-string image.tag="\$SOURCE_TAG" \ --set image.pullPolicy=IfNotPresent --set rtpengine.enabled=true \ --set rtpengine.image.repository=ghcr.io/sudheerkumarvatrapu/playsbc-rtpengine \ --set-string rtpengine.image.tag=3.0.0 --set rtpengine.hostNetwork=false \ --set playsbc.config.media_backend=rtpengine \ --set-string playsbc.config.rtpengine_url=udp://playsbc-playsbc-rtpengine:2223 \ --set observability.enabled=true </pre>	
COMMAND (CONTINUED)	SELECT + COPY
<pre> kubectl --context kind-playsbc -n playsbc rollout restart \ statefulset/playsbc-playsbc statefulset/playsbc-playsbc-rtpengine \ deployment/playsbc-playsbc-prometheus deployment/playsbc-playsbc-grafana kubectl --context kind-playsbc -n playsbc rollout status \ statefulset/playsbc-playsbc --timeout=240s kubectl --context kind-playsbc -n playsbc rollout status \ statefulset/playsbc-playsbc-rtpengine --timeout=240s kubectl --context kind-playsbc -n playsbc rollout status \ deployment/playsbc-playsbc-prometheus --timeout=240s kubectl --context kind-playsbc -n playsbc rollout status \ deployment/playsbc-playsbc-grafana --timeout=240s kubectl --context kind-playsbc -n playsbc get pods -o wide PYTHONPYCACHEPREFIX=/private/tmp/playsbc-private-pycache \ python3 tools/run_k8s_regression_job.py \ --all-profiles \ --playsbc-image "\$PLAYSBC_IMAGE" \ --set-playsbc-image --no-load-playsbc-image \ --build-runner-image --build-sipp-image --build-rtpengine-image \ --kind-load-images --set-rtpengine-image --kind-cluster playsbc </pre>	

Focused RFC 5359 Regression

The RFC 5359 plan covers all sections 2.1 through 2.18. Every multi-party regression must run three independent SIPp pods (A/B/C), including internal media and RTPengine variants. SIPp is not the production acceptance gate: standard-SIP real phones and softphones must also pass signaling, media, failure recovery, and cleanup in both supported device directions. See `docs/BUSINESS_CALLING_SERVICES.md` for the per-service status matrix. The paired internal/RTPengine profile catalog and compulsory A/B/C roles are defined in `docs/RFC5359_REGRESSION_PLAN.md`.

COMMAND**SELECT + COPY**

```
PYTHONPYCACHEPREFIX=/private/tmp/playsbc-public-pycache \  
python3 tools/run_public_foundation_regression.py  
  
PYTHONPYCACHEPREFIX=/private/tmp/playsbc-public-pycache \  
python3 tools/run_k8s_regression_job.py \  
  --profile rfc5359-call-hold-resume \  
  --profile rfc5359-call-hold-resume-rtpengine \  
  --profile rfc5359-call-hold-resume-tcp \  
  --profile rfc5359-call-hold-resume-tls \  
  --profile rfc5359-unattended-transfer \  
  --profile rfc5359-unconditional-forwarding \  
  --profile rfc5359-forwarding-on-busy \  
  --profile rfc5359-forwarding-on-no-answer \  
  --profile rfc5359-unattended-transfer-rtpengine \  
  --profile rfc5359-unconditional-forwarding-rtpengine \  
  --profile rfc5359-forwarding-on-busy-rtpengine \  
  --profile rfc5359-forwarding-on-no-answer-rtpengine \  
  --playsbc-image "$PLAYSBC_IMAGE" --set-playsbc-image --no-load-playsbc-image \  
  --build-runner-image --build-sipp-image --kind-load-images --kind-cluster playsbc
```

Evidence, Prometheus, And Grafana

COMMAND

SELECT + COPY

```
LATEST_REPORT=$(find logs/k8s-job -path '*/k8s-reports/latest.html' -type f | sort | tail -1)
: "${LATEST_REPORT:?No report found}"
open "$LATEST_REPORT"
```

Keep `latest.html` with its `evidence/` directory. If local links are blocked:

COMMAND

SELECT + COPY

```
python3 tools/serve_regression_report.py "$LATEST_REPORT"
```

COMMAND

SELECT + COPY

```
kubectl --context kind-playsbc -n playsbc port-forward \
  svc/playsbc-prometheus 9090:9090
```

COMMAND

SELECT + COPY

```
kubectl --context kind-playsbc -n playsbc port-forward \
  svc/playsbc-grafana 3000:3000
```

Open Prometheus at `http://127.0.0.1:9090`. Open Grafana at `http://127.0.0.1:3000` with admin / playsbc-lab.

Troubleshooting

- Root Dockerfile missing: always use `-f docker/playsbc.Dockerfile`.
- `--rollback-on-failure` unknown: use `--atomic` with the installed Helm.
- kind API refused: start Docker and `docker start playsbc-control-plane`.
- `ImagePullBackOff`: reload the exact image and retain `IfNotPresent`.
- Helm failure: inspect status, history, pod logs, and timestamp-sorted events.
- Broken report links: preserve the entire report directory or use its server.

COMMAND	SELECT + COPY
<pre>helm --kube-context kind-playsbc -n playsbc status playsbc helm --kube-context kind-playsbc -n playsbc history playsbc kubectl --context kind-playsbc -n playsbc get events --sort-by=.lastTimestamp kubectl --context kind-playsbc -n playsbc get pods -o wide</pre>	

Detailed Runbooks

These remain standalone and are not embedded in either generated guide: AKS .md, KUBERNETES_HELM_RUNBOOK .md, KUBERNETES_LOCAL .md, REAL_DEVICE_LAB .md, RTPENGINE_LOCAL .md, OBSERVABILITY .md, AI_VOICE_GATEWAY .md, BUSINESS_CALLING_SERVICES .md, and EVOLUTION_PLAN .md.